2º Trabalho de POOII

Aplicação de padrões de projeto ao sistema desenvolvido no Trabalho I

Documentação da aplicação dos padrões: **Service Layer**, **Factory Method**, **Strategy** e **Observer**.

Aluno	Gustavo Ynada Barbosa
Disciplina	Programação Orientada a Objetos II (POOII)
Padrões aplicados	Service Layer + Factory Method, Strategy e Observer

Resumo dos Padrões

Padrão	Categoria	Problema do código original
Service Layer	Arquitetural	Controllers acessavam DAO e continham as regras de negócio.
Factory Method	Criacional	`new ContaVIP` / `new ContaNormal` espalhados pelo controller e pelo DAO.
Strategy	Comportamental	Regra de retirada presa à hierarquia de Conta (método abstrato sobrescrito).
Observer	Comportamental	Telas atualizadas à mão (`atualizarTela()`) depois de cada operação.

Modo de Leitura

Cada padrão tem o mesmo roteiro do enunciado:

Identificação, **Situação anterior**, **Solução implementada** e **Justificativa e verificação**.

Em cada um há (a) um diagrama simples só com nomes e cores e (b) o exemplo de uma ação do sistema mostrando a **rota antes × depois**, passo a passo. Quando uma mesma ação passa por mais de um padrão, a rota é detalhada apenas no padrão principal. As cores são as mesmas em todo o documento:

View (FXML) | Controller | Service | DAO | Model | Factory Method | Strategy | Observer | Banco de dados

1. Diagnóstico do projeto original

O Trabalho I é um sistema bancário com três perfis: conta **Normal**, conta **VIP** (com limite e fatura de crédito) e **Administrador**. A interface é feita em JavaFX/FXML e os dados ficam em PostgreSQL, acessados pelo DAO. O sistema funcionava, mas foram registrado alguns problemas:

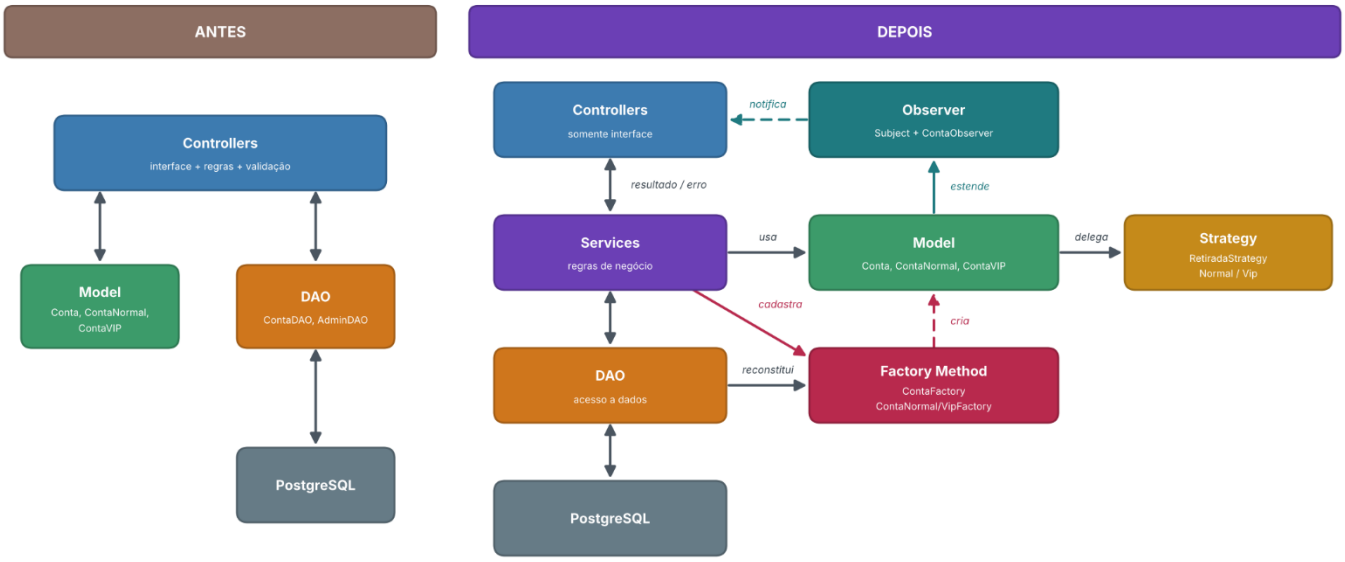
#	Situação observada no código original	Consequência	Tratado por
P1	Os controllers chamavam `ContaDAO`/`AdminDAO` diretamente e continham as regras de negócio: validação de CPF e senha, checagem de valor, de destinatário e de CPF duplicado. Ex.: `PainelNormalController.onTransferir()` fazia, sozinho, mais de 30 linhas de validação, busca no banco, transferência e gravação.	Interface, regra e acesso a dados misturados. difículdade de testar e de mexer.	Service Layer
P2	`PainelNormalController` e `PainelVipController` eram quase cópias (retirar, transferir, excluir conta). A regra de senha (4 a 26 caracteres) aparecia no controller e em `Conta.validarSenha`.	Cada mudança de regra exigia editar ambos os arquivos.	Service Layer

#	Situação observada no código original	Consequência	Tratado por
P3	A criação das contas era feita com if/else e `new ContaVIP(...)` / `new ContaNormal(...)` em `CadastroController` e em `ContaDAO.buscarContaPorCPF()`.	Interface e DAO conhecendo as models, um novo tipo de conta obrigaria a mexer nos dois pontos.	Factory Method
P4	A retirada era um método abstrato (`Conta.retirar`) sobrescrito em `ContaNormal` e `ContaVIP`; a lógica de uso de crédito ficava dentro da entidade VIP.	A regra de retirada fica presa à Strategy livrando o resto de sofrer alterações.	Strategy
P5	Depois de cada operação os painéis chamavam `atualizarTela()` manualmente e mantinham uma cópia local `conta`.	A tela só atualiza caso fosse chamada método por método no Service.	Observer

2. Visão geral das alterações

2.1 Arquitetura: antes × depois

Antes, o controller falava direto com o Model e com o DAO. Agora, nos fluxos que usam banco, a comunicação é **Controller → Service → DAO**, e o domínio (Model) passou a usar três padrões GoF.



- **Controller:** deixou de conhecer DAO e regras. Só lê a tela, chama o Service e mostra o resultado.
- **Service:** concentra validações e orquestra Model + DAO.
- **Model:** ganhou Factory, Strategy e Observer.

2.2 Estrutura de pacotes

Pacote	Antes	Depois
`banco.controller`	5 controllers + `Sessao`	5 controllers (sem `Sessao` e sem nenhum import de DAO)
`banco.service`	—	**novo:** `CadastroService`, `LoginService`, `TransacaoService`, `ContaService`, `PainelAdminService`
`banco.sessao`	—	**novo:** `Sessao`

Pacote	Antes	Depois
`banco.factory`	—	**novo:** `ContaFactory`, `ContaNormalFactory`, `ContaVipFactory`
`banco.strategy`	—	**novo:** `RetiradaStrategy`, `RetiradaNormalStrategy`, `RetiradaVipStrategy`
`banco.observer`	—	**novo:** `ContaObserver`, `Observer`
`banco.model`	`Admin`, `Conta`, `ContaNormal`, `ContaVIP`	`Conta`, `ContaNormal` e `ContaVIP` alteradas
`banco.dao`	`AdminDAO`, `ContaDAO`, `Conexao`	`ContaDAO` alterado
`banco` e telas	`Main`, `Launcher`, 5 arquivos FXML	Sem alteração

2.3 O que mudou, arquivo a arquivo

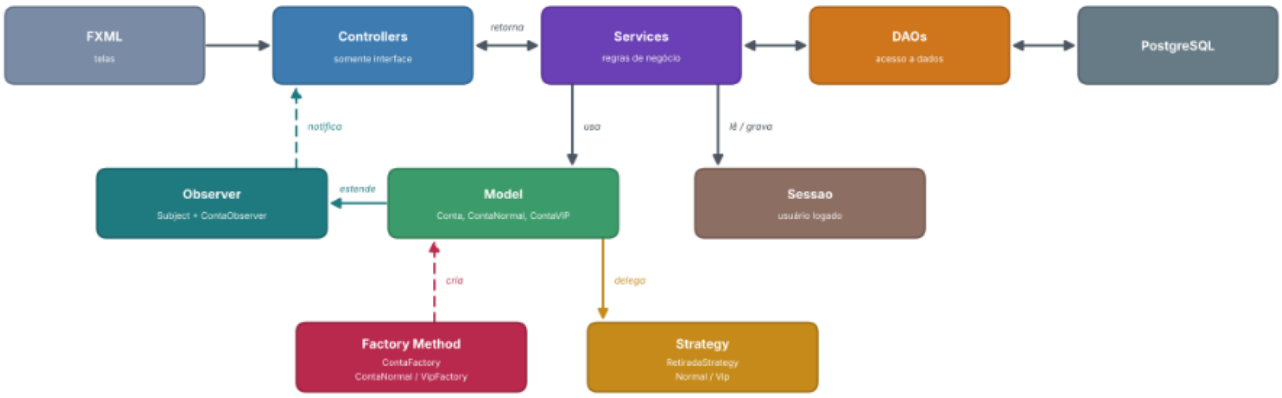
Arquivo	Como era	Como ficou
`CadastroController`	Validava nome, CPF e senha; consultava `ContaDAO`; escolhia `new ContaVIP` ou `new ContaNormal` com if/else; gravava no DAO.	Lê os campos e chama `CadastroService.cadastrarConta(...)`. Em caso de erro, mostra `e.getMessage()`.
`LoginController`	Consultava `AdminDAO` e `ContaDAO`, preenchia `Sessao` e usava `instanceof ContaVIP` para escolher a tela.	Chama `LoginService.autenticarUsuario()`, que devolve "ADM", "VIP" ou "NORMAL" O controller só troca de tela.
`PainelAdminController`	Validava valor e CPF, buscava a conta e gravava, tudo no controller.	Delega para `PainelAdminService.depositarSaldo()` e `cadastrarAdmin()`.
`PainelNormalController` e `PainelVipController`	Regras + DAO + `atualizarTela()` manual + cópia local `conta`.	Usam `TransacaoService` e `ContaService` Implementam `ContaObserver` e se atualizam em `contaAtualizada()`. Sem cópia local e atualizar tela manual.
`Sessao`	Em `banco.controller`, só dois campos públicos.	Em `banco.sessao`, com `limparSessao()`, `getContaVIP()`, `getContaNormal()`, `adicionarObserver()` e `removerObserver()`.
`Conta`	`retirar()` abstrato e sem notificação.	Estende `Observer`. Guarda uma `RetiradaStrategy`;. `retirar()`,`depositar()` e `setSaldo()` chamam `notificar()`.
`ContaNormal` / `ContaVIP`	Cada uma sobrescrevia `retirar()`	Passam a estratégia ao construtor (`RetiradaNormalStrategy` / `RetiradaVipStrategy`). `ContaVIP`: `podeCredito()` público, `setFaturaCredito()` notificando.
`ContaDAO`	`new ContaVIP(...)` para News. `ContaNormal(...)` ao ler do banco.	Usa `ContaFactory.obter(isVip)` e o factory method `reconstituirConta`.
`AdminDAO`, `Conexao`, `Admin`, `Main`, `Launcher`, FXML	—	Sem alteração.

3. Service Layer

padrão arquitetural · obrigatório

3.1 Identificação

Padrão	Service Layer (padrão arquitetural).
Finalidade no projeto	Concentrar as regras de negócio e a orquestração das operações (validar → aplicar a regra no Model → persistir pelo DAO), deixando o Controller só com a interface e o DAO só com o banco.
Classes / pacotes	`banco.service`: `CadastroService`, `LoginService`, `TransacaoService`, `ContaService`, `PainelAdminService`. Clientes: os cinco controllers. Colaboradores: DAOs, Model e `Sessao`.



Service	Operações	Responsabilidade
`CadastroService`	`cadastrarConta`	Valida nome, CPF e senhas; impede CPF duplicado; cria a conta (Factory) e grava.
`LoginService`	`autenticarUsuario`	Procura administrador e conta, confere a senha, preenche a `Sessao` e devolve o perfil; o controller só escolhe a tela.
`TransacaoService`	`validarSaque`, `fazerSaque`, `validarTransferencia`, `fazerTransferencia`, `validarFatura`, `pagarFatura`	Regras de saque, transferência e pagamento de fatura; aplica no Model e grava com o DAO.
`ContaService`	`excluirConta`, `faturaZerada`, `faturaPaga`	Confere a senha e remove a conta; regras de fatura para pagar e para excluir.
`PainelAdminService`	`depositarSaldo`, `cadastrarAdmin`	Depósito feito pelo administrador e criação de novos administradores, com validações.

3.2 Situação anterior

Problemas P1 e P2. O controller era tela, regra de negócio e cliente do banco. No trecho abaixo, `onTransferir()` valida tudo, busca o destinatário no `ContaDAO`, executa a transferência no Model e grava:

ANTES — PainelNormalController.onTransferir()

```
double valor = lerValor(dados.get()[0]);
String cpfDestino = Conta.tratamentoCpf(dados.get()[1]);
if (valor <= 0) { mostrarErro("Informe um valor válido"); return; }
if (!Conta.validarCpf(cpfDestino)) { mostrarErro("CPF do destinatário inválido"); return; }
if (cpfDestino.equals(conta.getCpf())) { mostrarErro("Você não pode transferir para você mesmo"); return; }
Conta destino = contaDAO.buscarContaPorCPF(cpfDestino);    // Controller -> DAO direto
if (destino == null) { mostrarErro("Destinatário não encontrado"); return; }
if (!confirmar("Confirmar transferência", ...)) return;
if (!conta.transferencia(destino, valor)) { mostrarErro("Saldo insuficiente"); return; }
if (!contaDAO.atualizar(conta) || !contaDAO.atualizar(destino)) {    // Controller -> DAO direto
```

```
        recarregar();
        mostrarErro("Erro ao salvar. Confira seu saldo e tente novamente");
        return;
    }
    atualizarTela();
```

3.3 Solução implementada

Toda operação que usa banco passou a seguir **Controller → Service → DAO**. O controller só lê os campos, chama o service e mostra a mensagem. As regras ficaram em `TransacaoService`: `validarTransferencia()` (valor, CPF, auto-transferência e destinatário existente) e `fazerTransferencia()` (aplica no Model e grava). Como o service lê a conta logada em `Sessao`, o mesmo código atende contas Normal e VIP.

DEPOIS — PainelNormalController.onTransferir() (Controller só delega)

```
double valor = lerValor(dados.get()[0]);
String cpfDestino = dados.get()[1];
try {
    TransacaoService service = new TransacaoService();
    String nomeDestino = service.validarTransferencia(valor, cpfDestino);   // regras + busca do destino
    if (!confirmar("Confirmar transferência", ...)) return;
    service.fazerTransferencia(valor, cpfDestino);   // Model + DAO
    mostrarSucesso("Transferência de " + moeda(valor) + " realizada");
} catch (Exception e) {
    mostrarErro(e.getMessage());
}
```

DEPOIS — TransacaoService.fazerTransferencia() (banco.service)

```
public void fazerTransferencia(double valor, String cpfDestino) throws Exception {
    Conta conta = Sessao.contaLogada;
    ContaDAO contaDAO = new ContaDAO();
    cpfDestino = Conta.tratamentoCpf(cpfDestino);
    Conta contaDestino = contaDAO.buscarContaPorCPF(cpfDestino);   // Service -> DAO
    if (!conta.transferencia(contaDestino, valor)) {               // regra no Model
        throw new Exception("Saldo insuficiente");
    }
    if (!contaDAO.atualizar(conta) || !contaDAO.atualizar(contaDestino)) {   // persistência
        throw new Exception("Erro ao salvar. Confira seu saldo e tente novamente");
    }
}
```

3.4 Exemplo de rota: Transferir dinheiro (conta Normal)

Cenário: o cliente informa um valor e o CPF de outro cliente e confirma. Cada caixa é um passo; a cor mostra a camada. O que antes era feito pelo controller agora é distribuído entre as camadas.



O que mudou nesta rota

- O controller deixou de falar com `ContaDAO` e com `Conta.transferencia()`. Agora só conhece `TransacaoService`.
- As regras saíram da tela (passo 2) e a gravação ficou no service (passos 5 a 7).
- No passo 6 a retirada usa o **Strategy** (seção 5) e, ao mudar o saldo, a tela é atualizada pelo **Observer** (seção 6). Por isso esses dois padrões não repetem este exemplo.

3.5 Justificativa e verificação

- **Benefício:** as regras existem uma única vez (Normal e VIP compartilham `TransacaoService` e `ContaService`), o controller fica simples e os services não dependem de JavaFX, o que permite testá-los sem abrir a interface.
- **Evidência no código:** nenhum controller importa `banco.dao`. Cada tela usa o service correspondente, conforme a tabela abaixo.

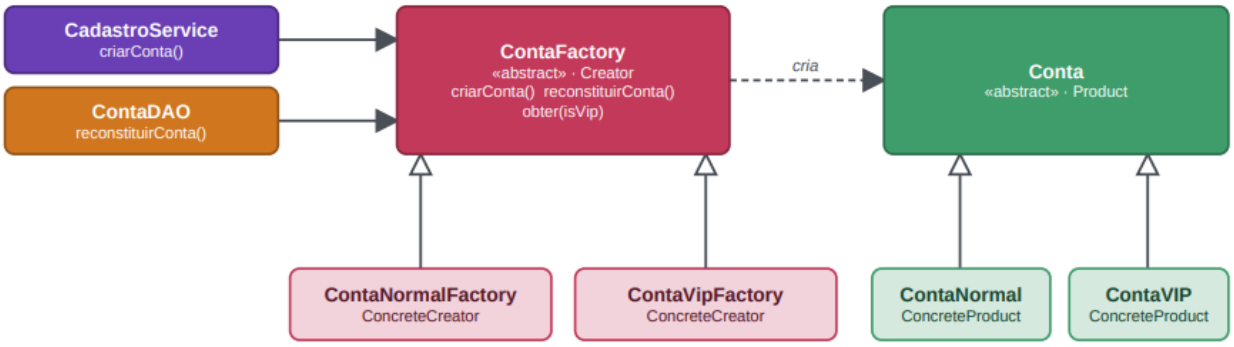
Controller	Service chamado	Operações
`CadastroController`	`CadastroService`	`cadastrarConta`
`LoginController`	`LoginService`	`autenticarUsuario`
`PainelAdminController`	`PainelAdminService`	`depositarSaldo`, `cadastrarAdmin`
`PainelNormalController`	`TransacaoService`, `ContaService`	`validarSaque`, `fazerSaque`, `validarTransferencia`, `fazerTransferencia`, `excluirConta`
`PainelVipController`	`TransacaoService`, `ContaService`	as mesmas + `validarFatura`, `pagarFatura`, `faturaZerada`, `faturaPaga`

4. Factory Method

GoF · criacional

4.1 Identificação

Padrão	Factory Method (Criacional).
Finalidade no projeto	Centralizar a criação dos objetos `Conta` (Normal ou VIP), escondendo as classes concretas de quem precisa de uma conta.
Classes / pacotes	`banco.factory`: `ContaFactory`, `ContaNormalFactory`, `ContaVipFactory`. Produtos em `banco.model`: `Conta`, `ContaNormal`, `ContaVIP`. Clientes: `CadastroService` e `ContaDAO`.



Papel no padrão	Classe
Creator	`ContaFactory` (abstrata), com os factory methods `criarConta(nome, cpf, senha)` e `reconstituirConta(nome, cpf, senha, saldo, limite, fatura)`
ConcreteCreator	`ContaNormalFactory` e `ContaVipFactory`
Product	`Conta` / `ContaNormal` e `ContaVIP`
Clientes	`CadastroService` (`criarConta`) e `ContaDAO` (`reconstituirConta`)

`criarConta` gera uma conta nova, com os valores iniciais do sistema (Normal: R$ 100; VIP: R$ 200 e limite de R$ 1.000). `reconstituirConta` monta a conta a partir de uma linha lida do banco (saldo, limite e fatura já persistidos). O método estático `obter(boolean)` apenas escolhe qual fábrica concreta usar.

4.2 Situação anterior

Problema P3: duas camadas diferentes decidiam qual classe instanciar, cada uma com seu `if/else`.

ANTES — CadastroController.onCadastrar() e ContaDAO.buscarContaPorCPF()

```
// CadastroController (interface criando objeto de domínio)
Conta nova;
if (tglVip.isSelected()) {
    nova = new ContaVIP(nome, cpf, senha);   // classe concreta no controller
} else {
    nova = new ContaNormal(nome, cpf, senha);
}
// ContaDAO (leitura do banco)
if (isVip) {
    return new ContaVIP(rs.getString("nome"), cpf, rs.getString("senha"), rs.getDouble("saldo"),
            rs.getDouble("limite_credito"), rs.getDouble("fatura_credito"));
} else {
    return new ContaNormal(rs.getString("nome"), cpf, rs.getString("senha"), rs.getDouble("saldo"));
}
```

4.3 Solução implementada

A decisão de qual classe criar passou para as fábricas. O cliente pede uma fábrica com `ContaFactory.obter(isVip)` e chama o factory method, sem conhecer `ContaVIP` nem `ContaNormal`.

DEPOIS — Creator abstrato e um ConcreteCreator

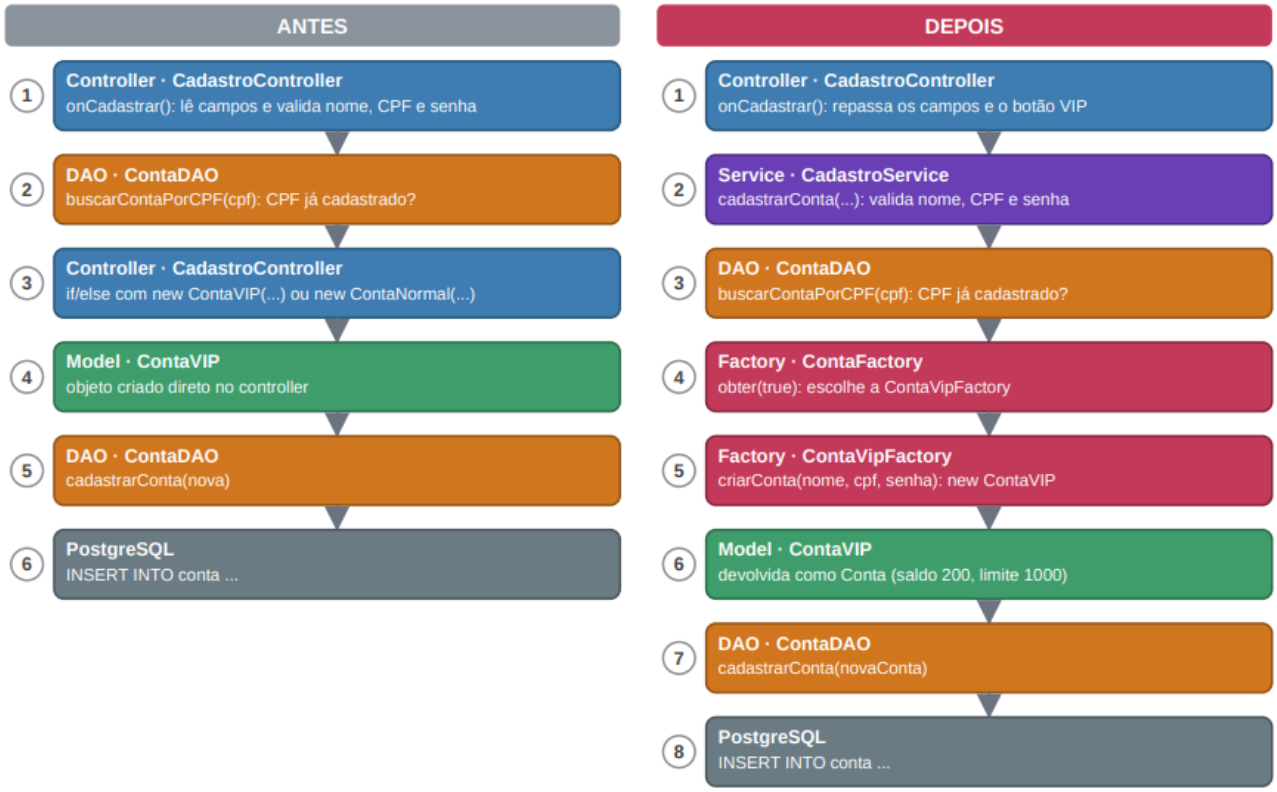
```
public abstract class ContaFactory {
    public abstract Conta criarConta(String nome, String cpf, String senha);          // factory method
    public abstract Conta reconstituirConta(String nome, String cpf, String senha,    // factory method
                                            double saldo, double limite, double fatura);
    public static ContaFactory obter(boolean isVip) {   // escolhe a fábrica concreta
        return isVip ? new ContaVipFactory() : new ContaNormalFactory();
    }
}
public class ContaVipFactory extends ContaFactory {
    @Override
    public Conta criarConta(String nome, String cpf, String senha) {
        return new ContaVIP(nome, cpf, senha);   // criação isolada aqui
    }
    @Override
    public Conta reconstituirConta(String nome, String cpf, String senha,
                                   double saldo, double limite, double fatura) {
        return new ContaVIP(nome, cpf, senha, saldo, limite, fatura);
    }
}
```

DEPOIS — Clientes das fábricas

```
// CadastroService.cadastrarConta()
Conta novaConta = ContaFactory.obter(isVIP).criarConta(nome, cpf, senha);
// ContaDAO.buscarContaPorCPF()
return ContaFactory.obter(isVip).reconstituirConta(
        rs.getString("nome"), cpf, rs.getString("senha"),
        rs.getDouble("saldo"), rs.getDouble("limite_credito"), rs.getDouble("fatura_credito"));
```

4.4 Exemplo de rota: Cadastrar uma conta VIP

Cenário: na tela de cadastro, o cliente preenche os dados, marca “Conta VIP” e clica em Cadastrar. A rota mostra onde o objeto `ContaVIP` nasce.



O que mudou nesta rota

- O `new` saiu do controller (que antes decidia com if/else): a escolha agora está nos passos 4 e 5 do DEPOIS, dentro das fábricas.
- O Service e o DAO só enxergam o tipo `Conta`. Na leitura do banco (login) o caminho é o mesmo, usando `reconstituirConta`.

4.5 Justificativa e verificação

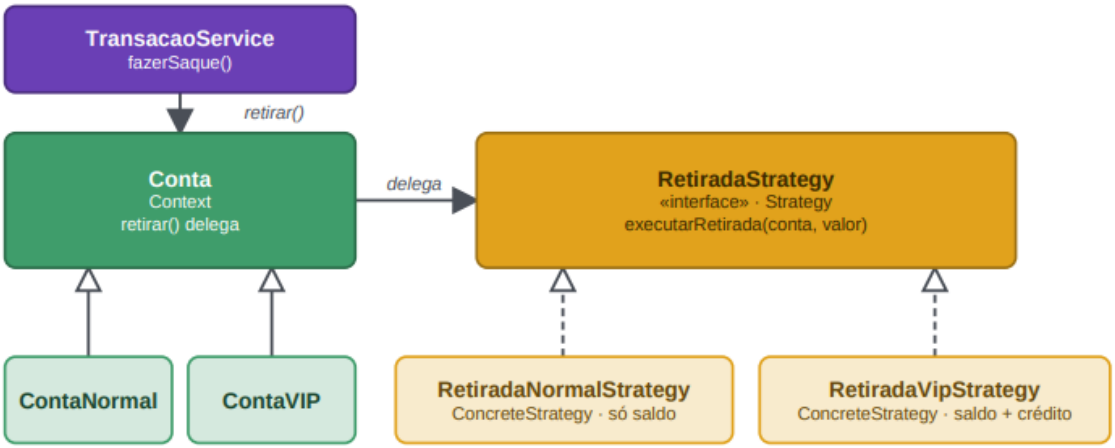
- **Benefício:** a criação fica em um só lugar. Um novo tipo de conta exige uma nova fábrica e um novo produto, sem alterar controller, service ou DAO.
- **Evidência no código:** `new ContaVIP(` e `new ContaNormal()` só aparecem dentro de `ContaVipFactory` e `ContaNormalFactory`. `CadastroController` não importa mais nenhuma classe de modelo nem de DAO. A fábrica é usada em dois pontos: `CadastroService.cadastrarConta()` e `ContaDAO.buscarContaPorCPF()`.

5. Strategy

GoF · comportamental

5.1 Identificação

Padrão	Strategy (Comportamental).
Finalidade no projeto	Isolar a regra de retirada em objetos próprios: a conta Normal só debita o saldo e a VIP usa o saldo e, se faltar, o crédito.
Classes / pacotes	`banco.strategy`: `RetiradaStrategy`, `RetiradaNormalStrategy`, `RetiradaVipStrategy`. Contexto: `Conta` (`banco.model`). Cliente: `TransacaoService`.



Papel no padrão	Classe
Context	`Conta`: guarda uma `RetiradaStrategy`. `retirar(valor)` chama ela
Strategy	`RetiradaStrategy` (interface): `executarRetirada(Conta, double)`
ConcreteStrategy	`RetiradaNormalStrategy` (debita se `checarSaldo`) e `RetiradaVipStrategy` (saldo e, se faltar, crédito)
Quem escolhe	`ContaNormal` e `ContaVIP`, passando a estratégia ao construtor de `Conta`
Cliente	`TransacaoService` (via `Conta.retirar()` e `Conta.transferencia()`)

5.2 Situação anterior

Problema P4: `retirar()` era abstrato em `Conta` e cada subclasse trazia sua regra. Funcionava, mas a política de retirada ficava amarrada à subclasse (a lógica de crédito, inclusive, dentro da entidade VIP): qualquer regra nova exigiria criar outra subclasse de `Conta` ou editar o `retirar()` existente.

ANTES — Conta, ContaNormal e ContaVIP

```
// Conta
public abstract boolean retirar(double valor);
// ContaNormal
public boolean retirar(double valor){
    if(checarSaldo(valor)){
        setSaldo(this.getSaldo() - valor);
        return true;
    }
    return false;
}
// ContaVIP (regra de crédito dentro da entidade)
public boolean retirar(double valor){
    if(valor <= 0) return false;
    if(valor <= this.getSaldo()){
        this.setSaldo(this.getSaldo() - valor);
        return true;
    } else{
        if(podeCredito(valor)){
            double creditoNecessario = valor - getSaldo();
```

```
            faturaCredito += creditoNecessario;
            this.setSaldo(0);
            return true;
        }
    }
    return false;
}
```

5.3 Solução implementada

O “como retirar” virou um objeto à parte. `Conta` deixou de ter regra de retirada e só delega à estratégia que recebeu no construtor. A regra da conta VIP foi movida, sem mudar de comportamento, para `RetiradaVipStrategy`.

DEPOIS — Strategy é escolhido a depender da classe

```
// Strategy
public interface RetiradaStrategy {
    boolean executarRetirada(Conta conta, double valor);
}
// Context: Conta
private RetiradaStrategy estrategiaRetirada;
public Conta(String nome, String cpf, String senha, double saldo, RetiradaStrategy estrategiaRetirada){
    /* ... */
    this.estrategiaRetirada = estrategiaRetirada;
}
public boolean retirar(double valor){
    return estrategiaRetirada.executarRetirada(this, valor);   // delega
}
// ContaVIP escolhe a estratégia no construtor (ContaNormal faz o mesmo com RetiradaNormalStrategy)
public ContaVIP(String nome, String cpf, String senha){
    super(nome, cpf, senha, 200.0, new RetiradaVipStrategy());
    this.limiteCredito = 1000;
    this.faturaCredito = 0;
}
```

DEPOIS — ConcreteStrategy da conta VIP

```
public class RetiradaVipStrategy implements RetiradaStrategy{
    public boolean executarRetirada(Conta conta, double valor){
        ContaVIP contaVip = (ContaVIP) conta;
        if(valor <= 0) return false;
        if(valor <= contaVip.getSaldo()){
            contaVip.setSaldo(contaVip.getSaldo() - valor);
            return true;
        } else{
            if(contaVip.podeCredito(valor)){
                double creditoNecessario = valor - contaVip.getSaldo();
                contaVip.setFaturaCredito(contaVip.getFaturaCredito() + creditoNecessario);   // usa crédito
                contaVip.setSaldo(0);
                return true;
            }
        }
        return false;
    }
}
```

5.4 Exemplo de rota: Retirar R$ 120 numa conta VIP com saldo de R$ 50

Cenário: conta VIP com saldo de R$ 50,00, limite de R$ 1.000,00 e fatura zerada. O cliente retira R$ 120,00. Faltam R$ 70,00, que devem ir para a fatura de crédito.



O que mudou nesta rota

- A decisão “usa só o saldo ou também o crédito?” saiu da entidade: agora o Service pede `retirar()` à conta e a conta delega à estratégia (passos 3 a 5), que devolve true ou false.
- Se a estratégia devolve false, o Service lança “Saldo insuficiente” e nada é gravado, se devolve true, o Service grava com o DAO (passos 6 e 7).
- A mesma rota vale para `transferencia()`, que chama `retirar()` por dentro.

5.5 Justificativa e verificação

- **Benefício:** a regra de retirada pode evoluir (ou ganhar uma regra nova) sem tocar em `Conta`: basta criar outra `RetiradaStrategy`. Cada regra fica em uma classe pequena e testável isoladamente.
- **Evidência no código:** `Conta.retirar()` deixou de ser abstrato; `ContaNormal` não tem mais `retirar()`; `ContaVIP` ficou sem `retirar()` e mantém `podeCredito()`, `pagarFatura()` e o estado do crédito. Todo saque e toda transferência passam por `Conta.retirar()` → estratégia.

.

6. Observer

GoF · comportamental

6.1 Identificação

Padrão	Observer (Comportamental).
Finalidade no projeto	Fazer os painéis Normal e VIP se atualizarem sozinhos quando saldo ou fatura da conta logada mudam, sem o controller precisar lembrar de chamar `atualizarTela()`.
Classes / pacotes	`banco.observer`: `Observer` e `ContaObserver`. `banco.model`: `Conta`, `ContaVIP`. `banco.controller`: `PainelNormalController`, `PainelVipController`. `banco.sessao`: `Sessao` (registro e remoção dos observers).

O registro é feito por `Sessao.adicionarObserver(this)` no `initialize()` e desfeito por `Sessao.removerObserver(this)` no `onSair()`.

Sobre o nome das classes: a classe base se chama `Observer`, mas, no vocabulário do GoF, ela exerce o papel de **Subject** (guarda a lista de interessados e dispara `notificar()`). O papel de Observer é da interface `ContaObserver`.

6.2 Situação anterior

Problema P5: cada painel guardava uma cópia local da conta e, depois de cada operação, chamava `atualizarTela()` na mão (e `recarregar()` se a gravação falhasse). Se alguma operação esquecesse a chamada, a tela ficava desatualizada.

ANTES — PainelVipController.onPagarFatura()

```
if (!conta.pagarFatura(valor)) {
    mostrarErro("Saldo insuficiente para pagar a fatura");
    return;
}
if (!contaDAO.atualizar(conta)) {
    recarregar();   // relê a conta do banco se der erro
    mostrarErro("Erro ao salvar. Tente novamente");
    return;
}
atualizarTela();   // chamada manual, repetida em cada operação
mostrarSucesso("Fatura paga: " + moeda(pago));
```

6.3 Solução implementada

A conta virou o **sujeito observável**: sempre que saldo ou fatura mudam (`depositar`, `setSaldo`, `setFaturaCredito`), ela chama `notificar()`. Os painéis Normal e VIP implementam `ContaObserver`, registram-se na conta logada via `Sessao` ao abrir e se removem ao sair.

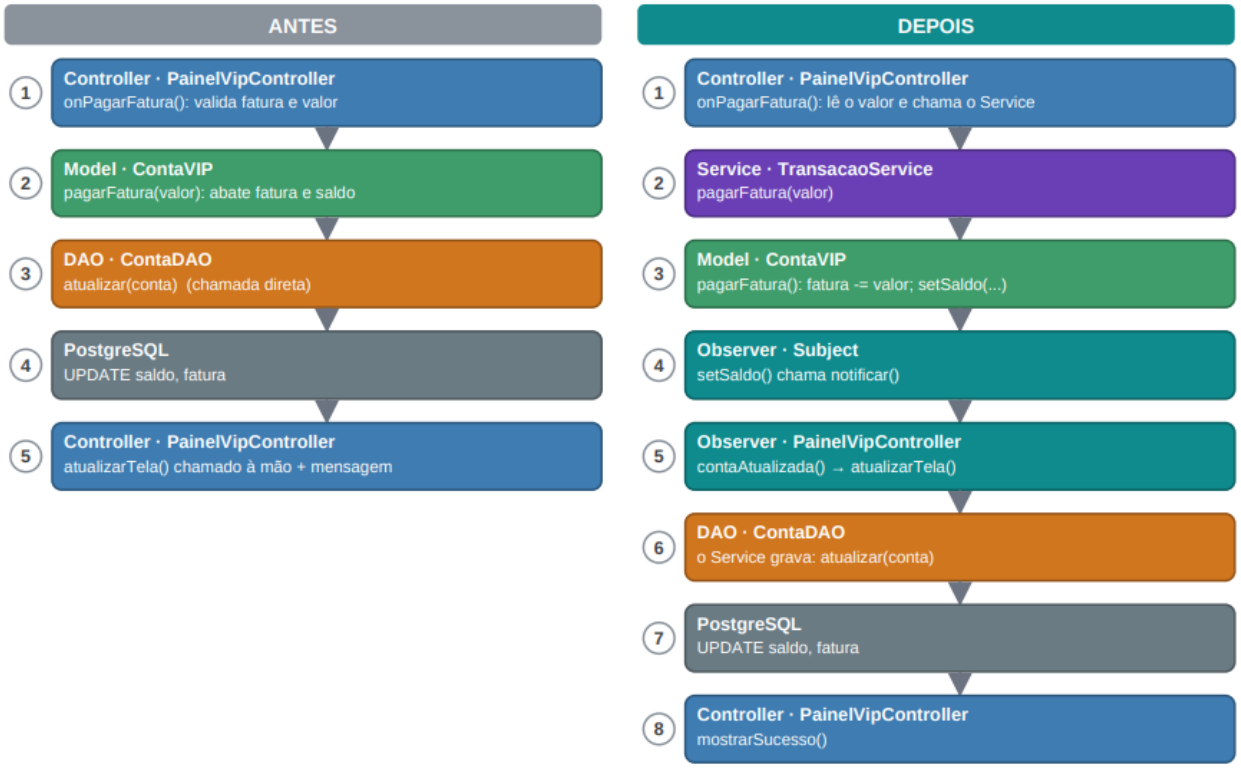
DEPOIS — Subject, ConcreteSubject e ConcreteObserver

```
// Subject (classe base de Conta)
public abstract class Observer {
    private final List<ContaObserver> observers = new ArrayList<>();
    public void adicionarObserver(ContaObserver observer) {
        if (!observers.contains(observer)) observers.add(observer);
    }
    public void removerObserver(ContaObserver observer) { observers.remove(observer); }
    protected void notificar() {
        for (ContaObserver observer : new ArrayList<>(observers)) {
            observer.contaAtualizada();
        }
    }
}
// ConcreteSubject: Conta e ContaVIP notificam quando o estado muda
public void setSaldo(double saldo){ this.saldo = saldo; notificar(); }    // Conta
public void setFaturaCredito(double faturaCredito){                       // ContaVIP
    this.faturaCredito = faturaCredito;
    notificar();
}
// ConcreteObserver: PainelVipController (o PainelNormalController é igual)
public class PainelVipController implements ContaObserver {
    @FXML private void initialize() { esconderMsg(); Sessao.adicionarObserver(this); atualizarTela(); }
    public void contaAtualizada() { atualizarTela(); }
    @FXML private void onSair() { Sessao.removerObserver(this); Sessao.limparSessao(); /* ... */ }
```

```
}
```

6.4 Exemplo de rota: Pagar a fatura de crédito (conta VIP)

Cenário: conta VIP com fatura de R$ 70,00 e saldo suficiente. O cliente clica em “Pagar fatura” e paga R$ 70,00. Essa rota não passa pelo Strategy, então mostra só o Observer em ação.



O que mudou nesta rota

- O passo 5 do ANTES (`atualizarTela()` manual) desapareceu: no DEPOIS, quem avisa a tela é o próprio Model, nos passos 4 e 5.
- Em `ContaVIP.pagarFatura()` a fatura é abatida antes de `setSaldo()`. Como é o `setSaldo()` que notifica, a tela já recebe saldo e fatura atualizados.
- A notificação acontece antes do DAO gravar (passo 6), pois o Observer reage à mudança do objeto em memória.

ContaVIP.pagarFatura() — a ordem importa

```
if(this.checarSaldo(valor)){
    faturaCredito -= valor;                  // 1) a fatura é abatida (sem notificar)
    this.setSaldo(this.getSaldo() - valor);  // 2) setSaldo() notifica: a tela já vê os dois valores
    return true;
}
```

6.5 Justificativa e verificação

- **Benefício:** qualquer mudança na conta, mesmo as feitas dentro do Strategy ou do Service, aparece na tela sem chamada manual; o controller não precisa saber quando atualizar.
- **Evidência no código:** nos dois painéis `atualizarTela()` só é chamado em `initialize()` e em `contaAtualizada()`; as operações (retirar, transferir, pagar fatura) não chamam mais. O `onSair()` remove o observer para a conta não notificar uma tela já fechada.

7. Roteiro de verificação no sistema

Duas sugestões de rotas que verificam os quatros padrões implementados no sistema:

#	O que fazer	O que observar	Padrão(ões)
1	Cadastrar uma conta VIP ou Normal	Controller chama o Service para fazer o cadastro, no Service terá o Factory que aplicará os conceitos da contaVIP ou contaNormal.	Service, Factory
2	VIP: retirar além do saldo, dentro do limite.	Saldo vai a zero, a fatura aumenta aplicando as regras do Strategy e a tela atualiza sozinha por conta do Observer.	Strategy, Observer

8. Conclusão

O sistema do Trabalho I manteve suas funcionalidades e ganhou quatro padrões integrados aos fluxos reais: **Service Layer** (organização em camadas Controller → Service → DAO), **Factory Method** (criação das contas), **Strategy** (regra de retirada) e **Observer** (atualização automática das telas).